

Monitoramento de Métricas TCP com eBPF SockOps

Um tutorial prático do kernel ao terminal

TCP-metrics Project

Observabilidade de Rede com eBPF

10 de junho de 2026

Sumário

- 1 Motivação e Visão Geral
- 2 Pré-requisitos e Ambiente
- 3 Programa eBPF SockOps
- 4 Scripts de Carga e Descarga
- 5 Leitor Python: `tcp_metrics_reader.py`
- 6 Métricas Coletadas
- 7 Métricas Coletadas
- 8 Comparação de Abordagens eBPF

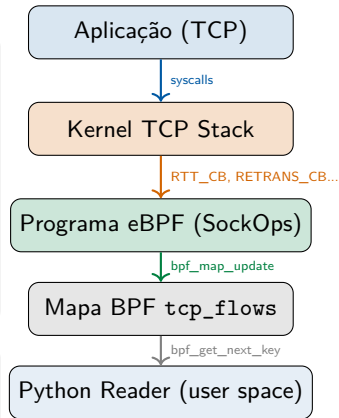
Por que monitorar métricas TCP no kernel?

O problema

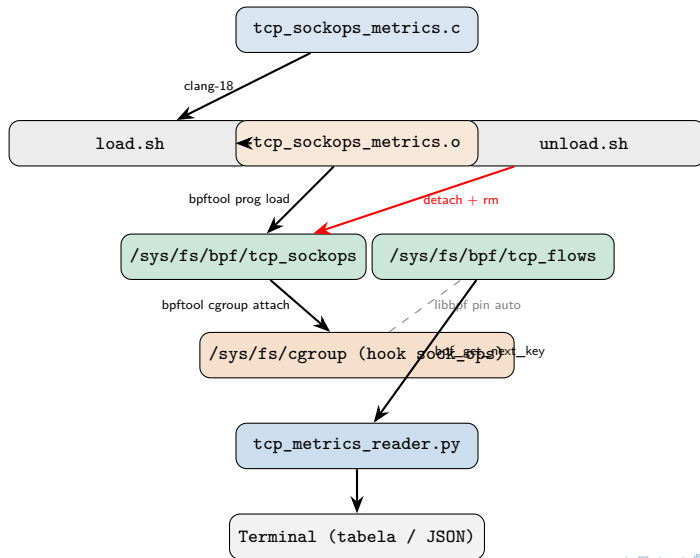
- Ferramentas clássicas (`ss`, `netstat`, `tcpdump`) capturam **snapshots** — não o histórico contínuo de cada fluxo
- `tcpdump` gera cópias de pacotes para o espaço de usuário: alto overhead
- Informações como **cwnd**, **RTT suavizado** e **ssthresh** só existem no `tcp_sock` do kernel

A solução: eBPF SockOps

- Código executado *dentro do kernel*, em pontos de hook TCP
- Zero cópia — acessa diretamente `bpf_sock_ops`
- Exporta dados por um **mapa BPF pinado** (`/sys/fs/bpf`)



Arquitetura geral do projeto



Arquivos do projeto

Arquivo	Descrição
<code>tcp_sockops_metrics.c</code>	Programa eBPF (tipo sockops) — coleta métricas por fluxo TCP via callbacks do kernel
<code>tcp_metrics_xdp.c</code>	Programa eBPF alternativo (tipo XDP) — captura em camada de pacotes (somente ingress)
<code>tcp_stats_sockkey.c</code>	Abordagem via kprobe — usa BCC para instrumentar funções do kernel
<code>load.sh</code>	Compila, carrega e anexa o programa BPF ao cgroup v2
<code>unload.sh</code>	Desanexa e remove o programa; limpa arquivos temporários
<code>tcp_metrics_reader.py</code>	Lê o mapa pinado e exibe métricas em tempo real (tabela ou JSON)
<code>monitor_tcp_metrics.py</code>	Leitor alternativo via bpftool (mais simples, para o XDP map)
<code>tcp_stats_monitor_sockkey.py</code>	Leitor dedicado para o mapa da abordagem kprobe
<code>llvm.sh</code>	Script auxiliar de instalação do LLVM/Clang

Pré-requisitos do sistema

Kernel e sistema

- Linux kernel ≥ 5.15 (recomendado 6.x)
- **cgroup v2** habilitado
- CONFIG_BPF_SYSCALL=y
- CONFIG_BPF_JIT=y
- CONFIG_SOCK_OPS=y

```
mount | grep cgroup2|
```

Verificar versão do kernel

```
uname -r  
# ex: 6.8.0-51-generic
```

Instalação de dependências

```
# Compilador BPF + bpftool  
sudo apt install \  
    clang-18 \  
    linux-tools-$(uname -r) \  
    bpftool  
  
# Python: leitura do mapa eBPF  
sudo apt install python3-bpfcc  
# ou (Fedora/RHEL):  
sudo dnf install python3-bcc  
  
# Teste de tráfego  
sudo apt install iperf3
```

Gerando o header `vmlinux.h`

Por que `vmlinux.h` é necessário?

O `vmlinux.h` contém *todos* os tipos e enums do kernel compilado — elimina a necessidade de incluir headers do kernel e garante que os valores de enum (como `BPF_SOCK_OPS_RTT_CB`) sejam exatamente os do kernel em execução.

Gerar (uma vez por boot/versão do kernel)

```
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
```

Diferença crítica — valores de enum

O programa `tcp_sockops_metrics.c` removeu definições manuais incorretas (BUG #2) e usa os valores reais exportados pelo `vmlinux.h`:

<code>BPF_SOCK_OPS_RTT_CB</code>	= 13 (kernel 6.x)
<code>BPF_SOCK_OPS_STATE_CB</code>	= 10
<code>BPF_SOCK_OPS_RETRANS_CB</code>	= 9

CO-RE: Compile Once, Run Everywhere

Com `vmlinux.h` + `bpf_core_read.h`, o objeto BPF compilado funciona em diferentes versões do kernel sem recompilação (desde que os campos existam).

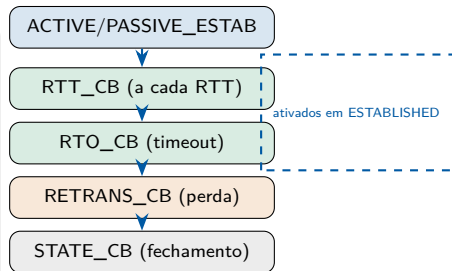
O que é SockOps?

Tipo de programa eBPF: BPF_PROG_TYPE_SOCK_OPS

- Executado em **eventos do ciclo de vida TCP**
- Recebe contexto `bpf_sock_ops *`: acesso direto a campos do socket como `snd_cwnd`, `srtt_us`, `bytes_acked`
- Anexado ao **cgroup v2** — abrange todos os sockets do cgroup
- Pode **ativar callbacks adicionais** por socket com `bpf_sock_ops_cb_flags_set()`

Vantagem sobre XDP/kprobe

Acessa métricas de controle de congestionamento diretamente do `tcp_sock` — sem parsear pacotes, sem overhead de cópia.



Estruturas de dados: chave e valor do mapa

tcp_flow_key — chave do mapa

```
1 struct tcp_flow_key {
2     union {
3         __u32 src_ip4;
4         __u32 src_ip6[4];
5     };
6     union {
7         __u32 dst_ip4;
8         __u32 dst_ip6[4];
9     };
10    __u16 src_port;
11    __u16 dst_port;
12    __u8  family;    // AF_INET ou AF_INET6
13    __u8  pad[3];    // alinhamento
14};
```

Union permite reutilizar a mesma estrutura para IPv4 e IPv6, economizando memória no mapa.

tcp_flow_stats — valor do mapa

```
1 struct tcp_flow_stats {
2     __u64 srtt_us;
3     __u32 cwnd;        // cwnd atual
4     __u32 cwnd_max;    // pico historico
5     __u32 ssthresh;
6     __u32 pkts_out;    // segmentos em voo
7     __u64 bytes_acked;
8     __u64 retransmissions;
9     __u64 last_bytes_acked;
10    __u64 last_segs_out;
11    __u64 timestamp_ns;
12};
```

cwnd_max rastreia o pico histórico da janela — útil para identificar a fase de slow-start.

Definição do mapa BPF

Mapa hash pinado

```
1 struct {  
2     __uint(type, BPF_MAP_TYPE_HASH);  
3     __uint(max_entries, 16384);  
4     __type(key, struct tcp_flow_key);  
5     __type(value, struct tcp_flow_stats);  
6     __uint(pinning, LIBBPF_PIN_BY_NAME);  
7 } tcp_flows SEC(".maps");
```

Pin automático pelo libbpf

O campo `LIBBPF_PIN_BY_NAME` faz o libbpf criar automaticamente o arquivo `/sys/fs/bpf/tcp_flows` ao carregar o programa — sem nenhuma chamada explícita.

Limite de 16.384 entradas

Produção: sem limpeza, as entradas se esgotam rapidamente. O programa remove entradas ao detectar `TCP_CLOSE` ou `TCP_CLOSE_WAIT` no `STATE_CB`.

Tipo escolhido: BPF_MAP_TYPE_HASH

- Lookup $O(1)$ por fluxo
- Chave composta (IP + porta + família)
- Alternativa: `LRU_HASH` (descarta entradas antigas automaticamente)

Construção da chave de fluxo: build_flow_key()

```
1 static __always_inline int
2 build_flow_key(struct bpf_sock_ops *skops,
3                struct tcp_flow_key *key)
4 {
5     __builtin_memset(key, 0, sizeof(*key));
6     __u32 family = skops->family;
7
8     if (family == AF_INET) {
9         key->src_ip4 = skops->local_ip4;
10        key->dst_ip4 = skops->remote_ip4;
11        key->family = AF_INET;
12        key->src_port = bpf_ntohs(
13            (__u16)skops->local_port);
14        // porta nos 16 bits ALTOS (assimetria do
15            kernel)
16        key->dst_port = bpf_ntohs(
17            (__u16)(skops->remote_port >> 16));
18    } else if (family == AF_INET6) {
19        key->src_ip6[0] =
20            BPF_CORE_READ(skops, local_ip6[0]);
21        // ... (demais words)
22        key->family = AF_INET6;
23    } else { return -1; }
24    return 0;
25 }
```

Assimetria de portas no kernel

local_port: host byte order, 16 bits baixos.

remote_port: **network** byte order, 16 bits **altos**.

Por isso: remote_port » 16 antes do bpf_ntohs().

Restrição do verifier BPF

O verifier proíbe aritmética em ponteiros de contexto (ctx). Por isso os campos local_ip6[] são lidos elemento a elemento com BPF_CORE_READ — nunca via ponteiro intermediário modificado.

Função principal: ativação de callbacks

```
150 SEC("sockops")
151 int tcp_metrics(struct bpf_sock_ops *skops)
152 {
153     if (skops->family != AF_INET &&
154         skops->family != AF_INET6)
155         return 0;
156
157     // Ativa callbacks no estabelecimento
158     if (skops->op ==
159         BPF SOCK OPS ACTIVE_ESTABLISHED_CB ||
160         skops->op ==
161         BPF SOCK OPS PASSIVE_ESTABLISHED_CB) {
162         bpf_sock_ops_cb_flags_set(skops,
163             BPF SOCK OPS RTT_CB_FLAG |
164             BPF SOCK OPS STATE_CB_FLAG |
165             BPF SOCK OPS RETRANS_CB_FLAG |
166             BPF SOCK OPS RTO_CB_FLAG);
167     }
```

Melhoria #1 (bug original)

Sem chamar `bpf_sock_ops_cb_flags_set()` no estabelecimento, o kernel **nunca** invoca RTT_CB nem STATE_CB — o programa ficaria sem coletar métricas.

Callbacks ativados

RTT_CB_FLAG	a cada RTT
STATE_CB_FLAG	mudança de estado
RETRANS_CB_FLAG	retransmissão
RTO_CB_FLAG	timeout

Função principal: coleta de métricas e limpeza

```
189 // Limpeza ao fechar conexao
190 if (skops->op == BPF SOCK OPS STATE_CB) {
191     if (skops->args[1] == BPF_TCP_CLOSE ||
192         skops->args[1] == BPF_TCP_CLOSE_WAIT) {
193         bpf_map_delete_elem(&tcp_flows, &key);
194         return 0;
195     }
196 }
197
198 // Atualiza metricas no RTT_CB / RTO_CB
199 if (skops->op == BPF SOCK OPS RTT_CB || ...) {
200     stats->srtt_us = skops->srtt_us >> 3;
201     stats->ssthresh = skops->snd_ssthresh;
202     stats->cwnd = skops->snd_cwnd;
203     if (skops->snd_cwnd > stats->cwnd_max)
204         stats->cwnd_max = skops->snd_cwnd;
205 }
206
207 // Retransmissao direta
208 if (skops->op == BPF SOCK OPS RETRANS_CB) {
209     __sync_fetch_and_add(
210         &stats->retransmissions, 1);
211 }
```

Melhoria #2 — limpeza automática

Sem `bpf_map_delete_elem()` no fechamento, as 16.384 entradas se esgotam em produção e novos fluxos não conseguem ser inseridos.

`srtt_us` » 3

O kernel armazena o RTT suavizado multiplicado por 8 (deslocado 3 bits). A divisão converte para microssegundos reais.

Estimativa indireta de retransmissões

Se `segs_out` cresceu mas `bytes_acked` ficou igual entre dois `RTT_CB`s, há indício de retransmissão — utilizado como fallback quando o `RETRANS_CB` ainda não estava ativo.

load.sh — Compilar, carregar e anexar

```
#!/bin/bash
# 1. Compilar o programa BPF
clang-18 -O2 -g \
  -target bpf \
  -D__TARGET_ARCH_x86 \
  -I. \
  -c tcp_sockops_metrics.c \
  -o tcp_sockops_metrics.o

# 2. Remover pins antigos
rm -f /sys/fs/bpf/tcp_sockops \
  /sys/fs/bpf/tcp_flows

# 3. Carregar e pinar o programa
bpftool prog load \
  tcp_sockops_metrics.o \
  /sys/fs/bpf/tcp_sockops \
  type sockops

# 4. Anexar ao cgroup v2
bpftool cgroup attach \
  /sys/fs/cgroup sock_ops \
  pinned /sys/fs/bpf/tcp_sockops multi
```

Etapas explicadas

- 1 **Compilação:** clang-18 com target bpf gera bytecode eBPF a partir do C
- 2 **Limpeza:** evita conflito com pins de execuções anteriores
- 3 **Carga:** bpftool prog load passa o objeto pelo verifier do kernel e pina o fd em /sys/fs/bpf/tcp_sockops
- 4 **Attach:** hook sock_ops no cgroup raiz; flag multi permite coexistir com outros programas BPF

Requer root

Todos os comandos precisam de privilégios de superusuário (sudo).

unload.sh — Descarga segura com opções

Opções disponíveis

```
# Limpeza interativa (pede confirmacao)
sudo ./unload.sh

# Força limpeza sem confirmacao
sudo ./unload.sh --force

# Apenas exibe status atual
sudo ./unload.sh --status

# Limpa mas mantém .o e vmlinux.h
sudo ./unload.sh --keep-files
```

Funções do script

check_root()	verifica \$EUID
is_program_loaded()	bpftool prog show pinned
is_attached_to_cgroup()	grep no output do cgroup show
detach_from_cgroup()	detach + verificação
unload_bpf()	remove pin + verificação
cleanup_files()	remove .o e vmlinux.h
show_current_status()	lista attachments e pins

Robustez

O script usa `set -e` e `set -u` — qualquer falha aborta a execução imediatamente, evitando estados parcialmente limpos.

Sequência de limpeza completa

- 1 bpftool cgroup detach (com flag multi, depois sem)
- 2 rm /sys/fs/bpf/tcp_sockops_metrics (remove pin)
- 3 Remove .o e vmlinux.h
- 4 Verifica outros programas BPF com nome similar

Visão geral do leitor Python

O que faz

- Abre o mapa pinado em `/sys/fs/bpf/tcp_flows` via **libbcc**
- Itera sobre todas as entradas com `bpf_get_next_key`
- Decodifica chaves IPv4/IPv6 com **ctypes** (estruturas C espelhadas)
- Exibe tabela colorida ou JSON a cada `-interval` segundos

Modos de saída

- **Tabela** (padrão): limpa a tela a cada ciclo, destaques por cor
- **JSON**: uma linha por ciclo, adequado para pipelines (`jq`, `Prometheus`, etc.)

Parâmetros da linha de comando

<code>-map PATH</code>	caminho do mapa pinado
<code>-interval N</code>	segundos entre ciclos (padrão: 1)
<code>-top N</code>	máx. fluxos exibidos (padrão: 15)
<code>-filter-ip IP</code>	filtra por endereço IP
<code>-json</code>	saída em JSON
<code>-sort CAMPO</code>	ordena por <code>srtt/cwnd/retrans/bytes</code>

Destaques visuais

- **Amarelo**: `RTT > 100 ms`
- **Vermelho**: `retransmissões > 0`
- **Verde**: `cwnd = cwnd_max` e `> 10` (crescendo)

Estruturas ctypes — espelhando o kernel em Python

```
1 class TcpFlowKeyIPv4(ctypes.Structure):
2     _fields_ = [
3         ("src_ip4", ctypes.c_uint32),
4         # union: preenche 4x u32
5         ("_pad_src", ctypes.c_uint32 * 3),
6         ("dst_ip4", ctypes.c_uint32),
7         ("_pad_dst", ctypes.c_uint32 * 3),
8         ("src_port", ctypes.c_uint16),
9         ("dst_port", ctypes.c_uint16),
10        ("family", ctypes.c_uint8),
11        ("pad", ctypes.c_uint8 * 3),
12    ]
13
14 class TcpFlowStats(ctypes.Structure):
15     _fields_ = [
16         ("srtt_us", ctypes.c_uint64),
17         ("cwnd", ctypes.c_uint32),
18         ("cwnd_max", ctypes.c_uint32),
19         ("ssthresh", ctypes.c_uint32),
20         ("pkts_out", ctypes.c_uint32),
21         ("bytes_acked", ctypes.c_uint64),
22         ("retransmissions", ctypes.c_uint64),
23         ("last_bytes_acked", ctypes.c_uint64),
24         ("last_segs_out", ctypes.c_uint64),
25         ("timestamp_ns", ctypes.c_uint64),
26    ]
```

Correspondência exata com o C

Os campos e tamanhos das estruturas Python **devem ser idênticos** às estruturas C do programa eBPF. Qualquer divergência de padding ou ordem gera dados corrompidos na leitura.

Por que padding no IPv4?

A estrutura C usa union {u32 src_ip4; u32 src_ip6[4];} — no Python, é simulada com src_ip4 + 3 palavras de padding para garantir que o tamanho total seja o mesmo que o IPv6 (16 bytes).

Iteração sobre o mapa BPF

```
1 def read_map(map_fd: int) -> dict:
2     flows = {}
3     key_buf = (ctypes.c_uint8 *
4                 ctypes.sizeof(TcpFlowKeyIPv6))()
5     next_buf = (ctypes.c_uint8 *
6                  ctypes.sizeof(TcpFlowKeyIPv6))()
7     val_buf = (ctypes.c_uint8 *
8                 ctypes.sizeof(TcpFlowStats))()
9
10    ctypes.memset(key_buf, 0, sizeof(key_buf))
11
12    while True:
13        ret = bcc_lib.bpf_get_next_key(
14            map_fd,
15            ctypes.cast(key_buf, c_void_p),
16            ctypes.cast(next_buf, c_void_p),
17        )
18        if ret != 0:
19            break # fim da iteracao
20
21        bcc_lib.bpf_lookup_elem(
22            map_fd,
23            ctypes.cast(next_buf, c_void_p),
24            ctypes.cast(val_buf, c_void_p),
25        )
26        # decodifica familia e cria flow_id...
27        ctypes.memmove(key_buf, next_buf, ...)
28    return flows
```

Padrão de iteração BPF

- 1 Inicializa key_buf com zeros (chave nula = início)
- 2 bpf_get_next_key: retorna a próxima chave em next_buf
- 3 bpf_lookup_elem: lê o valor para next_buf
- 4 Copia next_buf para key_buf e repete
- 5 Quando ret $\neq$ 0: fim do mapa

Atenção: TOCTOU

Entre o get_next_key e o lookup_elem, o programa eBPF pode ter deletado a entrada (conexão fechada). O código ignora o erro do lookup_elem sem crashar.

Tabela de métricas

Coluna	Campo BPF	Descrição
RTT(μ s)	skops->srtt_us	RTT suavizado; programa divide por 8 (»3)
CWND	skops->snd_cwnd	Janela de congestionamento atual (segmentos MSS)
CWND_MAX	-	Maior valor de cwnd observado na conexão
IN_FLIGHT	skops->packets_out	Segmentos enviados ainda não confirmados
SSTHRESH	skops->snd_ssthresh	Limiar de slow-start (∞ quando $\geq 0xFFFF$)
BYTES_ACK	skops->bytes_acked	Total acumulado de bytes confirmados
RETRANS	-	Contagem por RETRANS_CB e estimativa indireta
IDADE	timestamp_ns	Tempo desde a última atualização da entrada

Vermelho

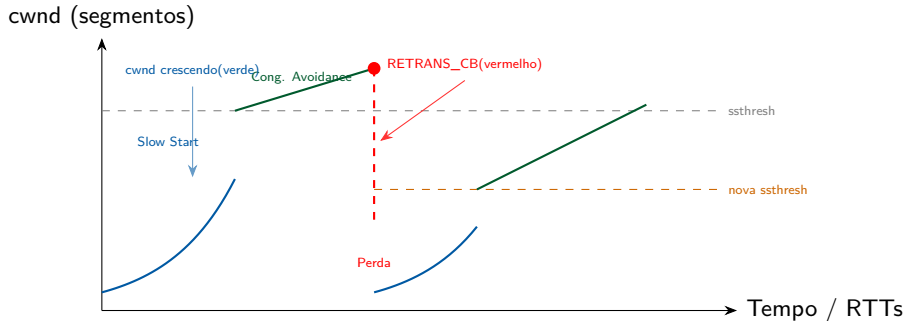
Retransmissões > 0: possível congestionamento ou perda.

RTT > 100 ms: latência elevada ou bufferbloat.

Verde

cwnd=cwnd_max e > 10: crescimento saudável da janela.

Controle de Congestionamento TCP: contexto das métricas



Três abordagens: SockOps vs XDP vs kprobe

Característica	SockOps	XDP	kprobe (BCC)
Nível de acesso	tcp_sock (L4)	Pacotes raw (L3)	Qualquer função
RTT suavizado	✓ direto	✗ indireto	✓ direto
CWND	✓ direto	✗ N/A	✓ direto
Retransmissões	✓ preciso	! aprox.	✓ preciso
Overhead	✓ mínimo	! médio	! médio
Attach	cgroup v2	Interface	Símbolo kernel
Portabilidade	CO-RE	Menor	Dependente da versão
Arquivo	tcp_sockops_metrics.c	tcp_metrics_xdp.c	tcp_stats_sockkey.c

Limitação do XDP

XDP observa apenas tráfego de entrada. Como ACKs normalmente não passam pelo mesmo hook, o RTT permanece zero em condições normais. Para testes, utilize `iperf3 -b 500k` ou `xdpgeneric` com tráfego loopback.

Abordagem XDP: tcp_metrics_xdp.c

Compilar e carregar

```
# Compilar
sudo clang -O2 -g -target bpf \
  -D__TARGET_ARCH_x86 \
  -c tcp_metrics_xdp.c \
  -o tcp_metrics_xdp.o

# Carregar na interface
sudo ip link set dev enp1s0 xdp \
  obj tcp_metrics_xdp.o sec xdp

# Descarregar
sudo ip link set dev enp1s0 xdp off

# Verificar
sudo ip link show dev enp1s0

# Ler mapa
sudo bpftool map dump \
  pinned /sys/fs/bpf/xdp/globals/flows -p
```

Normalização da chave de fluxo

O XDP vê pacotes em *uma direção* (ingress). Para agregar o fluxo bidirecional, a chave é **normalizada** por ordem de IP/porta:

```
1  if (src_ip < dst_ip ||
2      (src_ip == dst_ip &&
3        src_port < dst_port)) {
4      fk.src_ip = src_ip;
5      fk.dst_ip = dst_ip;
6  } else {
7      fk.src_ip = dst_ip;
8      fk.dst_ip = src_ip;
9  }
```

Detecção de retransmissões

Usa um mapa auxiliar `seq_ts` (sequence → timestamp). Se o mesmo número de sequência aparecer duas vezes, é retransmissão.

Abordagem kprobe: tcp_stats_sockkey.c

Hooks instrumentados

```
1 // Enviando dados
2 int kprobe__tcp_sendmsg(
3     struct pt_regs *ctx,
4     struct sock *sk, ...) { ... }
5
6 // Retransmissao
7 int kprobe__tcp_retransmit_skb(
8     struct pt_regs *ctx,
9     struct sock *sk, ...) { ... }
10
11 // Mudanca de estado TCP
12 int kprobe__tcp_set_state(
13     struct pt_regs *ctx,
14     struct sock *sk, int state) { ... }
```

Métricas coletadas via kprobe

- pkts_sent, bytes_sent: contadores acumulados
- srtt_us / rtt_us: lidos do tcp_sock com bpf_probe_read_kernel
- cwnd: tp->snd_cwnd
- cc_ops_ptr: ponteiro para o algoritmo de congestionamento ativo
- last_state: último estado TCP

Portabilidade

kprobes dependem de símbolos do kernel compilados — nomes de funções podem mudar entre versões. SockOps + CO-RE é mais portátil.

Fluxo completo de uso

1. Gerar vmlinux.h

```
bpftool btf dump \
  file /sys/kernel/btf/vmlinux \
  format c > vmlinux.h
```

2. Carregar o programa

```
sudo ./load.sh
```

3. Monitorar

```
# Tabela padrao (sort: RTT)
sudo python3 tcp_metrics_reader.py

# Intervalo 2s, top 20, sort cwnd
sudo python3 tcp_metrics_reader.py \
  --interval 2 --top 20 --sort cwnd

# Filtrar por IP
sudo python3 tcp_metrics_reader.py \
  --filter-ip 192.168.1.100
```

4. Gerar tráfego de teste

```
# Transferencia de 60s para
# observar crescimento do cwnd
iperf3 -c <IP_DESTINO> \
  -t 60 -i 1 &

sudo python3 tcp_metrics_reader.py \
  --sort cwnd
```

5. Remover ao terminar

```
# Com confirmacao
sudo ./unload.sh

# Sem confirmacao
sudo ./unload.sh --force

# Apenas ver status
sudo ./unload.sh --status
```


Estrutura de diretórios em tempo de execução

Sistema de arquivos BPF

```
/sys/fs/bpf/  
|-- tcp_sockops      # fd do programa  
'-- tcp_flows        # mapa hash pinado
```

Cgroup v2

```
/sys/fs/cgroup/  
'-- (raiz) <- ponto de attach  
    hook: sock_ops, flag: multi
```

Verificar pins e attachments

```
# Ver programas carregados  
sudo bpftool prog show \  
    pinned /sys/fs/bpf/tcp_sockops  
  
# Ver attachments do cgroup  
sudo bpftool cgroup show \  
    /sys/fs/cgroup
```

Saída esperada após load.sh

```
$ sudo bpftool prog show \  
    pinned /sys/fs/bpf/tcp_sockops  
42: sock_ops name tcp_metrics  
    tag a1b2c3d4e5f60718  
    loaded_at 2024-01-15T10:30:00  
    xlated 1024B jited 768B  
    map_ids 7
```

```
$ sudo bpftool cgroup show \  
    /sys/fs/cgroup
```

ID	AttachType	AttachFlags	Name
42	sock_ops	multi	tcp_metrics

Diagnóstico rápido

Se o mapa tcp_flows não aparecer após load.sh, verifique se o vmlinux.h foi gerado e se o /sys/fs/bpf/ está montado (bpffs).

Race condition: BPF_NOEXIST e lookup-after-update

```
1 stats = bpf_map_lookup_elem(  
2     &tcp_flows, &key);  
3  
4 if (!stats) {  
5     // BPF_NOEXIST evita sobrescrever  
6     // entrada criada por outro CPU  
7     // entre o lookup e o update  
8     bpf_map_update_elem(  
9         &tcp_flows, &key,  
10        &zero, BPF_NOEXIST);  
11  
12    // Segundo lookup necessario  
13    // update nao retorna ponteiro  
14    stats = bpf_map_lookup_elem(  
15        &tcp_flows, &key);  
16    if (!stats)  
17        return 0;  
18 }
```

Por que BPF_NOEXIST?

Em sistemas multi-CPU, dois núcleos podem executar o programa BPF simultaneamente para o mesmo fluxo. Sem BPF_NOEXIST, o segundo CPU sobrescreveria a entrada criada pelo primeiro, perdendo dados já coletados.

Por que dois lookups?

bpf_map_update_elem() não retorna ponteiro para o valor. O segundo bpf_map_lookup_elem() é necessário para obter o ponteiro e atualizar os campos in-place.

Operações atômicas e o verifier BPF

Contagem atômica de retransmissões

```
1 // Operacao atomica - segura em multi-CPU
2 __sync_fetch_and_add(
3     &stats->retransmissions, 1);
```

Restrições do verifier BPF

O verifier rejeita programas que:

- Acessam memória fora dos bounds
- Deferenciam ponteiros não verificados
- Têm loops potencialmente infinitos
- Usam aritmética em ponteiros de contexto

BPF_CORE_READ para IPv6

Para ler arrays de contexto BPF sem violar as restrições do verifier:

```
1 // Correto: offset constante
2 key->src_ip6[0] =
3     BPF_CORE_READ(skops,
4                     local_ip6[0]);
5
6 // ERRADO: ponteiro modificado
7 // __u32 *p = skops->local_ip6;
8 // key->src_ip6[0] = p[0]; // FALHA
```

__always_inline

Funções auxiliares como `build_flow_key()` devem ser `__always_inline` — o verifier BPF não suporta chamadas de função normais em programas do tipo sockops.

Pontos-chave do projeto

O que aprendemos

- **SockOps** é a abordagem mais direta para métricas TCP: acesso nativo ao `tcp_sock`
- `bpf_sock_ops_cb_flags_set()` **deve** ser chamado no estabelecimento — sem isso nenhum `RTT_CB` é disparado
- O mapa deve fazer limpeza no `STATE_CB` para evitar esgotamento
- `vmlinux.h` + CO-RE garantem portabilidade entre versões do kernel
- `BPF_NOEXIST` evita race condition no insert multi-CPU

Extensões possíveis

- Substituir `BPF_MAP_TYPE_HASH` por `LRU_HASH` (sem limpeza manual)
- Adicionar ringbuffer (`BPF_MAP_TYPE_RINGBUF`) para streaming de eventos
- Exportar para Prometheus via `-json` | `promtail`
- Suporte a IPv6 completo no leitor Python (já parcialmente implementado)
- Alertas em tempo real para RTT alto ou burst de retransmissões

Referência	URL
BPF sockops — kernel docs	https://www.kernel.org/doc/html/latest/bpf/prog_sockops.html
bpftool man page	https://man7.org/linux/man-pages/man8/bpftool.8.html
libbpf CO-RE guide	https://nakryiko.com/posts/bpf-core-reference-guide/
BCC Python bindings	https://github.com/iovisor/bcc

Obrigado!

Monitoramento de Métricas TCP com eBPF SockOps

```
sudo ./load.sh  
sudo python3 tcp_metrics_reader.py -sort cwnd  
sudo ./unload.sh -force
```

Código disponível em <https://github.com/nerds-ufes/Prog-Networks-2026/tree/main/lab-04> · Compatível com
kernel 6.x · Ubuntu/Debian/Fedora